

Final Exam — Sample B — Solutions

Software Engineering — UATX — Spring 2026

Question 1

T4 — React State & Effects (7.1) · Trace-through

This component renders once on mount, then the user clicks the button exactly once. Give the full console output in order, and answer the questions below.

```
function Likes() {
  const [likes, setLikes] = useState(3);

  useEffect(() => {
    console.log("effect:", likes);
  }, [likes]);

  function onClick() {
    setLikes(likes + 1);
    setLikes(likes + 1);
    console.log("handler:", likes);
  }

  return <button onClick={onClick}>{likes}</button>;
}
```

1. List everything the console logs, in order, from the initial mount through the one click.
2. After that single click, what number does the button show — 4 or 5? Explain why.
3. The dev wants one click to add 2 (so the button goes to 5). Give the one-line change and say why it works.

SOLUTION

1. effect: 3 (effects run after the first render) → then on the click, handler: 3 (the handler runs synchronously and reads this render's snapshot, where `likes === 3`) → then after the re-render, effect: 4 (the `[likes]` dep changed `3 → 4`).
2. The button shows **4**. Both `setLikes(likes + 1)` calls read the same render's snapshot (`likes === 3`), so both compute 4; React batches them into one re-render with `likes = 4`. The calls don't stack — state is a per-render snapshot, not a live variable.
3. Use the functional updater form: `setLikes(l => l + 1)` (both lines). Each updater receives the latest **queued** value (`3 → 4 → 5`), so one click ends at **5** and the effect logs effect: 5. It works because the updater doesn't depend on the stale snapshot `likes`.

Question 2

T1 — JavaScript & TypeScript (5.1) · Trace-through

The script runs to completion. Give the value printed at A, B, C, and D, then answer the follow-up.

```

const nums = [3, 1, 2];
const sorted = nums.sort((a, b) => b - a);
console.log("A:", nums, sorted); // ____

const prices = [
  { item: "pen", cents: 150 },
  { item: "pad", cents: 320 },
  { item: "pen", cents: 150 },
];
const total = prices.reduce((s, p) => s + p.cents, 0);
console.log("B:", total); // ____

const names = ["ann", "bob", "cat"];
const upper = names.map(n => n.toUpperCase());
console.log("C:", names, upper); // ____

console.log("D:", [1, 2, 3] + [4, 5], typeof NaN, 0.1 + 0.2 === 0.3); // ____

```

1. Give A, B, C, and D.
2. One of these lines quietly mutates a variable you might not expect. Which one, and how would you avoid the surprise?

SOLUTION

1. **A:** [3, 2, 1] [3, 2, 1]. `sort` sorts **in place** and returns the **same** array, so `sorted` is just another name for `nums`; both print the mutated array. **B:** 620 (150 + 320 + 150). **C:** ["ann", "bob", "cat"] ["ANN", "BOB", "CAT"]. `map` returns a new array and never touches the original. **D:** "1,2,34,5" "number" false. `+` on two arrays coerces both to strings ("1,2,3" + "4,5"); `typeof NaN` is "number"; `0.1 + 0.2` is 0.30000000000000004, so `=== 0.3` is false.
2. Line A — `sort` mutated `nums`, not just `sorted`. Copy first:
`const sorted = [...nums].sort((a, b) => b - a)` leaves `nums` untouched.

Question 3

T2 — Async, fetch & races (5.2) · Spot the bug

Your agent produced this live-search box. It works when you type slowly on a good connection. Find at least three defects; for each, one sentence on what goes wrong and one on the fix.

```

const box = document.querySelector("#q") as HTMLInputElement;
const out = document.querySelector("#out") as HTMLUListElement;

box.addEventListener("input", async () => {
  const res = await fetch("/api/search?q=" + box.value);
  const hits = await res.json();
  out.innerHTML = "";
  for (const h of hits) {
    const li = document.createElement("li");
    li.innerHTML = h.name;
  }
});

```

```

        out.append(li);
    }
});

```

SOLUTION

1. **No `res.ok` check.** `fetch` only rejects on a network failure, not on a 4xx/5xx, so an error response flows into `res.json()` (throwing a `SyntaxError`) or renders as if it were results. Guard with `if (!res.ok) throw new Error(...)` inside a `try/catch`.
2. **Out-of-order responses (the race).** Every keystroke starts a request and they can resolve in any order, so a slow "ca" can land after the newer "cat" and overwrite the screen with stale results. Cancel the in-flight request each keystroke with an `AbortController`, and/or ignore a response whose query no longer matches `box.value`.
3. **`li.innerHTML = h.name` is an XSS sink.** Search results rendered as HTML let a crafted name run script. Use `li.textContent = h.name`.

Bonus: it fires on **every** keystroke (debounce it, e.g. 250 ms), and it doesn't `encodeURIComponent(box.value)`, so a space or `&` corrupts the query string.

Question 4

T9 — Web Security Basics (9.2) · Spot the bug

Your agent produced this endpoint to read a direct message. It authenticates the caller and parameterizes its query. Read it carefully.

```

@app.get("/api/messages/{message_id}")
def get_message(message_id: int, user: User = Depends(current_user)):
    with engine.connect() as conn:
        row = conn.execute(
            text("SELECT * FROM messages WHERE id = :id"),
            {"id": message_id},
        ).mappings().first()
    if row is None:
        raise HTTPException(404)
    return dict(row)

```

1. The endpoint authenticates the caller. Why is that not enough here?
2. It has an access-control hole. Describe it, give a concrete request that exploits it, and give the fix.
3. `SELECT *` then `return dict(row)` creates a second, quieter problem. Name it and the fix.

SOLUTION

1. `Depends(current_user)` only proves **who** is calling (authentication). It never checks that this caller is allowed to see **this** message (authorization) — and that per-object check is exactly the line you have to write yourself. Authn ≠ authz.

2. **IDOR (insecure direct object reference).** WHERE `id = :id` ties the row to no user, so any authenticated user can enumerate ids: user A sends GET `/api/messages/777` for a DM between users B and C, the query matches, and A reads someone else's private message. Fix: scope ownership **in the query** —
`... WHERE id = :id AND (sender_id = :uid OR recipient_id = :uid) with :uid = user.id` — and return **404** (not 403) on no match so you don't even confirm the id exists.
3. **Over-exposure / leaking columns.** SELECT `* + dict(row)` ships every column to the client, including fields never meant to be public (internal flags, the other user's id, moderation/soft-delete columns). Select an explicit column list and return through a Pydantic response model that whitelists only the fields the client should see.

Question 5

T3 — React: Components & Rendering (6.1) · Specification

Feature request to your agent: “Show the feed of posts.” The first cut is below and it looks fine in a demo with a handful of posts. Before you let it ship, write the spec you'd hand back. Do not write code.

```
function Feed({ posts }: { posts: Post[] }) {
  return (
    <ul>
      {posts.map(p => (
        <li key={p.id}>
          <strong>{p.author}</strong>: {p.message}
          <span>{p.replies.length} replies</span>
        </li>
      ))}
    </ul>
  );
}
```

1. The rendering / display states this component must handle — name them.
2. Two things it gets wrong or omits for real data (look hard at empty results and partially-loaded posts).
3. One product decision that is ambiguous, and the call you would make.

SOLUTION

1. Empty feed (a real “no posts yet” state, not a bare empty `<ul>`); a single post vs many; a post with **0 replies, exactly 1 reply (singular!), and many** — “1 reply” not “1 replies”; very long messages (wrap, clamp, or “read more”); missing/optional fields (deleted author → null); and a large feed (it renders one `<li>` per post forever — needs pagination).
2. Any two: (a) an empty array renders an empty list with no message; (b) `p.replies.length` assumes every post carries a loaded `replies` array, but a list/feed endpoint usually omits

nested data, so a real payload throws

Cannot read properties of undefined (reading 'length') — prefer a flat

`p.replyCount ?? 0`. (Also: this takes `posts` as a prop, so **who** fetches them and owns the loading/error state is unspecified, and the sort order isn't defined.)

3. A real ambiguity, e.g.: all posts or paginated? newest-first or oldest-first? full message or truncated with “read more”? Pick one and state it (e.g. “newest-first, 20 per page, messages clamped to 4 lines”).

Question 6

T7 — CI/CD with GitHub Actions (8.2) · Code review

A teammate opens a PR adding this CI workflow. The repo is a FastAPI backend plus a React/TypeScript frontend (with Vitest and Playwright tests under `frontend/`), deployed on Railway. The team believes that once this merges, CI protects `main`.

```
# .github/workflows/ci.yml
name: CI
on:
  push:
    branches: [main]
jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt
      - run: pytest
      - run: echo "Deploying with ${ secrets.RAILWAY_TOKEN }}" && railway up
```

1. The team thinks merging to `main` is gated by this check. Give two reasons it isn't, and the fix.
2. Given a React + Vitest + Playwright frontend, what does this workflow never test? Why is that “green check” misleading?
3. Two things are wrong with the last step. Name both and say what you'd do instead.

SOLUTION

1. (a) It triggers on push to `main` only — it runs **after** code has already landed, and never on pull requests, so it can't block a bad merge. Add `pull_request`. (b) A workflow merely **existing** gates nothing; you must turn on branch protection (Settings → Branches → require the status check) so `main` rejects a red check. The workflow is a check; branch protection is the gate.
2. It never sets up Node or runs `npm ci` / `npm run build` / `vitest` / Playwright, and never runs lint/typecheck (`ruff`, `pyright`). So every frontend bug — and all the Playwright tests

the team wrote — sail through. A green check that only ran the backend gives false confidence: tests that don't run in CI protect nothing.

3. (a) `echo "...${{ secrets.RAILWAY_TOKEN }}"` prints the secret straight into the build log (public on a public repo) — never echo a secret; pass it via `env:` and let it be read, not logged. (b) CI shouldn't deploy at all here: Railway already auto-deploys on push to `main`, and putting the deploy token in CI widens the blast radius. Remove the deploy step; let Railway deploy, gated by the now-required check.

Question 7

T5 — Cloud Deployment (7.2) · Short design

Your agent set the BBS up for production on Railway with a Supabase Postgres. Relevant pieces of the setup are below. On `push to main`, Railway runs `pip install -r requirements.txt` and then `uvicorn app:app`. The frontend is built on the developer's laptop and the resulting `frontend/dist` is committed to the repo. There are no migrations; the developer edits the Supabase tables by hand in the dashboard.

```
# app.py (excerpt)
app = FastAPI()
app.mount("/", StaticFiles(directory="frontend/dist", html=True))

@app.get("/api/posts")
def list_posts():
    ...

DATABASE_URL = "postgresql://app:hunter2@db.supabase.co:5432/postgres"
engine = create_engine(DATABASE_URL)
```

1. After deploy, every call to `/api/posts` returns a 404 instead of your JSON — your handler never runs. Why, and what's the fix? (It's the same-origin static-serving trick, done wrong.)
2. There's a leaked secret here. Identify it, say how to inject it properly, and say what else you must do because it was already committed.
3. The build-and-schema process has two problems that cause “works on my machine” and dev/prod drift. Name both and give the right approach for each.

SOLUTION

1. `app.mount("/", StaticFiles(...))` is registered **before** the API routes, and a mount at `/` is a catch-all that matches every path — including `/api/posts`. Starlette matches routes in registration order, so the static mount answers first: it looks for a file at `api/posts`, doesn't find one, and 404s — your handler never runs. Fix: register the API routes first and mount the static catch-all **last**. (Done right, serving the React build from the same FastAPI app means same origin, so you also need no CORS.)
2. `DATABASE_URL` (with the DB password) is hardcoded in `app.py`, which is committed to the repo — a leaked credential readable by anyone with the repo. Read it from

`os.environ["DATABASE_URL"]` , set the value in Railway's dashboard, and keep a gitignored `.env` for local dev. Because it's already in git history, treat it as compromised and **rotate the database password**.

3. (a) Committing a laptop-built `frontend/dist` ships an un-reviewed, possibly stale bundle — build the frontend on Railway as part of deploy (`npm ci && npm run build`) so the deployed artifact comes from the committed source. (b) Hand-editing Supabase tables keeps the schema out of git and lets dev and prod drift apart — put schema changes under version control via Alembic migrations (`alembic upgrade head` on deploy) or a checked-in `schema.sql`.